

Week 2 - Day 7b: Recursion

Recursion is a method of solving problems where the overall solution depends on *solutions to smaller instances* of the same problem (subproblems). In the context of programming, this would involve *a function calling itself* with different arguments that describe the subproblem. See the following pseudo-code example of a recursive function

```
def recursive_function(parameters):  
    if base_case_condition(parameters):  
        return base_case_value  
    return recursive_function(modified_parameters)
```

Remember, although recursion can be an effective approach for many problems, it's not always the best choice. Recursive solutions are usually short to write, though often less efficient than iterative solutions (i.e. those that use `for` or `while` loops).

Functions in Python have a recursion limit, usually set to 1000. If you exceed this limit, Python will raise a `RecursionError`. You can check your recursion limit with `sys.getrecursionlimit()`, and change it with `sys.setrecursionlimit()`, but it's generally not a good idea to change the limit.

Exercise 1: Sum of Digits []

Define a recursive function called `sum_digits(n)` which takes an int `n` as an argument and returns the sum of the digits of `n`. Assume `n >= 0`. For example,

```
sum_digits(111) == 3    # because 1 + 1 + 1 == 3  
sum_digits(123) == 6    # because 1 + 2 + 3 == 6  
sum_digits(505) == 10   # because 5 + 0 + 5 == 10
```

Hint: Use the `%` and `//` operators

6.1 Implement `sum_digits(n)` recursively.

```
# TODO: Your code here  
def sum_digits(n):  
    pass  
  
# Here are some test cases to test your solution  
assert sum_digits(111) == 3  
assert sum_digits(123) == 6  
assert sum_digits(505) == 10  
assert sum_digits(123456789) == 45  
assert sum_digits(0) == 0  
  
print(f"Test cases passed! [ ]")
```

Exercise 2: Factorial []

Consider a recursive function called `factorial(n)`. It takes a integer $n > 0$ and returns the factorial ($n!$) of that number. Remember that

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1$$

Some examples are:

```
factorial(0) == 0
factorial(1) == 1
factorial(3) == 6
factorial(5) == 120
factorial(10) == 3628800
```

7.1 Implement `factorial(n)` recursively.

```
# TODO: Your code here
def factorial(n):
    pass

# Here are some test cases to test your solution
assert factorial(0) == 1
assert factorial(1) == 1
assert factorial(2) == 2
assert factorial(3) == 6
assert factorial(5) == 120

print(f"Test cases passed! [ ]")
```

Exercise 3: Palindrome Check []

Consider a recursive function called `check_palindrome(s)` which takes a string `s` and returns `True` if the word is a palindrome, and `False` otherwise. Remember that a palindrome is a word that reads the same forwards and backwards.

For example:

```
check_palindrome("racecar") == True
check_palindrome("hello") == False
check_palindrome("level") == True
check_palindrome("python") == False
```

8.1 Implement `check_palindrome(s)` recursively.

```
# TODO: Your code here
def check_palindrome(s):
    pass
```

```
# Here are some test cases to test your solution
assert check_palindrome("racecar") == True
assert check_palindrome("hello") == False
assert check_palindrome("level") == True
assert check_palindrome("python") == False

print(f"Test cases passed! 🎉")
```

Exercise 4: Count Digits []

Consider a recursive function called `count_digits(n)` which takes a positive integer `n` and returns the count of digits in `n`. For example:

```
count_digits(0) == 1
count_digits(123) == 3
count_digits(987654321) == 9
```

9.1 Implement `count_digits(n)` recursively.

```
# TODO: Your code here
def count_digits(n):
    pass

# Here are some test cases to test your solution
assert count_digits(0) == 1
assert count_digits(123) == 3
assert count_digits(987654321) == 9

print(f"Test cases passed! 🎉")
```

Exercise 5: Sum Range []

Consider a recursive function called `sum_range(a, b)` which takes two positive integers `a` and `b` and returns the sum of the numbers in between `a` and `b`, both including. For example:

```
sum_range(0, 0) == 0
sum_range(1, 5) == 15    # (1 + 2 + 3 + 4 + 5)
sum_range(10, 15) == 75  # (10 + 11 + 12 + 13 + 14 + 15)
sum_range(5, 10) == 45   # (5 + 6 + 7 + 8 + 9 + 10)
```

10.1 Implement `sum_range(a, b)` recursively

```
# TODO: Your code here
def sum_range(a, b):
    pass

# Here are some test cases to test your solution
assert sum_range(0, 0) == 0
```

```
assert sum_range(1, 5) == 15
assert sum_range(10, 15) == 75
assert sum_range(5, 10) == 45

print(f"Test cases passed! 🎉")
```

Congratulations 🎉